



GamesUP - Rapport technique final

Auteur : Valentin Bour

Version 1.0 - 21 août 2026

Périmètre : API Spring Boot, persistance MySQL, service de recommandation FastAPI et
exécution Docker Compose

Compétences : C.14, C.16, C.17 et C.18

Synthèse exécutive

GamesUP est désormais structuré comme une plateforme de vente de jeux de société composée d'une API publique Spring Boot, d'une base MySQL et d'un service FastAPI interne chargé de la recommandation collaborative. Le client Angular attendu par l'énoncé consomme l'API mais ne fait pas partie du périmètre de code livré dans ce dépôt. L'API Spring concentre l'authentification, les autorisations, les règles métier et les transactions ; FastAPI ne reçoit que des identifiants techniques et des signaux de préférence.

Le travail a remplacé un prototype fortement couplé par des composants organisés autour des responsabilités métier. Les données sont persistées par JPA/Hibernate, le schéma est piloté par quatre migrations Flyway, les entrées et sorties HTTP sont portées par des DTO validés et les erreurs suivent le format ProblemDetail. La sécurité est stateless avec JWT, BCrypt et deux rôles, CLIENT et ADMIN.

La campagne finale du backend Spring compte **113 tests réussis**. Le rapport JaCoCo mesure **1 212 lignes couvertes sur 1 298, soit 93,37 %**, au-dessus du seuil bloquant de 70 %. Le service Python implémente un KNN item-item avec distance cosinus, repli par popularité pour le démarrage à froid, artefact persistant versionné et contrôle de compatibilité au chargement.

Correspondance avec les compétences évaluées

Compétence	Démonstration dans le projet	Preuves principales
C.14 - Architecture	Architecture distribuée, modèle de données, composants, flux de bout en bout	Diagrammes d'architecture, classes, composants et séquence
C.16 - Qualité logicielle	SOLID, DTO, validation, migrations, transactions et erreurs homogènes	Packages Spring, ports, services applicatifs, OpenAPI et Flyway
C.17 - Tests et sécurité	Tests unitaires et d'intégration Spring, seuil JaCoCo, JWT, rôles et protection du service interne	Code/gamesUP/target/site/jacoco/index.html, pom.xml, workflow CI
C.18 - Machine learning	Données d'interaction, KNN cosinus, démarrage à froid, persistance et stratégie de réentraînement	Code/CodeApiPython/gamesup_api/, contrat Spring/FastAPI

Sommaire

- État initial et trajectoire de transformation
- C.14 - Architecture et modèle de données
- C.16 - Conception, qualité et persistance
- C.17 - Sécurité, tests et intégration continue
- C.18 - Recommandation KNN
- Démonstration Docker Compose
- Regard critique et améliorations
- Traçabilité des preuves

1. État initial et trajectoire de transformation

Défauts du prototype observé

Le point de départ remplissait une fonction de démonstration mais pas les exigences d'une application maintenable et sécurisée. Le diagnostic initial a identifié les défauts suivants :

- GameController mélangeait exposition HTTP, accès JDBC, SQL, configuration et traitement d'erreurs ;
- les objets du domaine n'étaient pas de véritables entités JPA et leurs champs étaient directement exposés ;
- Spring Security était désactivé, les identifiants MySQL étaient codés en dur et aucun DTO ne séparait le contrat HTTP du domaine ;
- la suite Java ne contenait qu'un test de chargement du contexte ;
- le service FastAPI renvoyait une liste statique de trois recommandations, sans apprentissage KNN ;
- l'absence de migrations, de rôles et de couverture mesurée rendait les évolutions risquées.

Ces constats expliquent la trajectoire retenue : isoler le domaine, introduire des services applicatifs, rendre les dépendances techniques remplaçables, sécuriser les contrats, fiabiliser la base et seulement ensuite brancher la recommandation.

Résultat et périmètre

La livraison finale couvre le backend demandé. Elle comprend les cas d'usage d'inscription et de connexion, la consultation du catalogue, le profil, la liste de souhaits, les avis, les commandes, l'administration des utilisateurs, du catalogue, du stock, des avis et des commandes, puis l'entraînement et la consultation des recommandations. Le contrat de l'API est publié dans `Code/gamesUP/docs/openapi.yaml`.

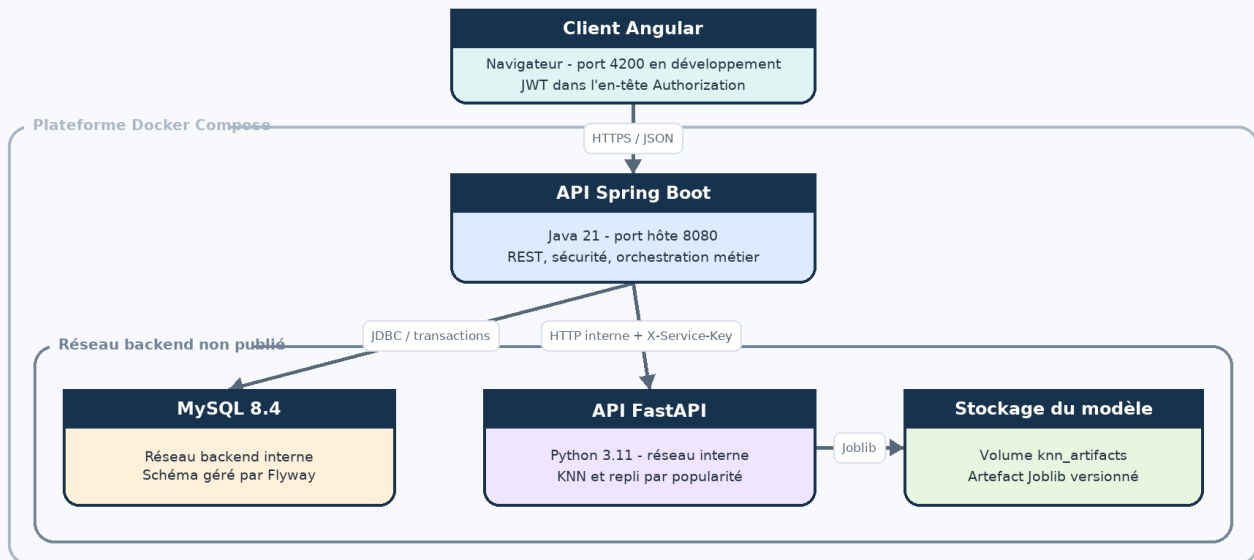
Le frontend Angular est représenté comme consommateur externe dans les diagrammes, conformément à l'architecture cible. Aucun code Angular n'étant présent dans le dépôt, ce rapport ne lui attribue ni écrans, ni tests, ni état de livraison.

2. C.14 - Architecture de la solution

Vue de déploiement

Architecture de déploiement GamesUP

Angular reste un client externe ; seul Spring est publié par Docker Compose.



Source : compose.yaml, configurations Spring et FastAPI

Architecture de déploiement Angular, Spring Boot, MySQL, FastAPI et stockage du modèle

L'unique port publié par compose.yaml est celui de Spring Boot, par défaut 8080. MySQL et FastAPI appartiennent au réseau Docker backend, déclaré interne avec le sous-réseau 172.28.0.0/24. Spring appartient aussi au réseau edge, ce qui lui permet d'être le point d'entrée. Les contrôles de santé ordonnent le démarrage : Spring attend MySQL et FastAPI, puis Flyway valide et applique le schéma.

L'architecture respecte deux frontières de confiance. D'une part, Angular ne contacte jamais directement la base ni le moteur de recommandation. D'autre part, FastAPI vérifie à la fois la provenance réseau de la requête et l'en-tête X-Service-Key. L'artefact Joblib est conservé dans le volume knn_artifacts, indépendamment du cycle de vie d'un conteneur.

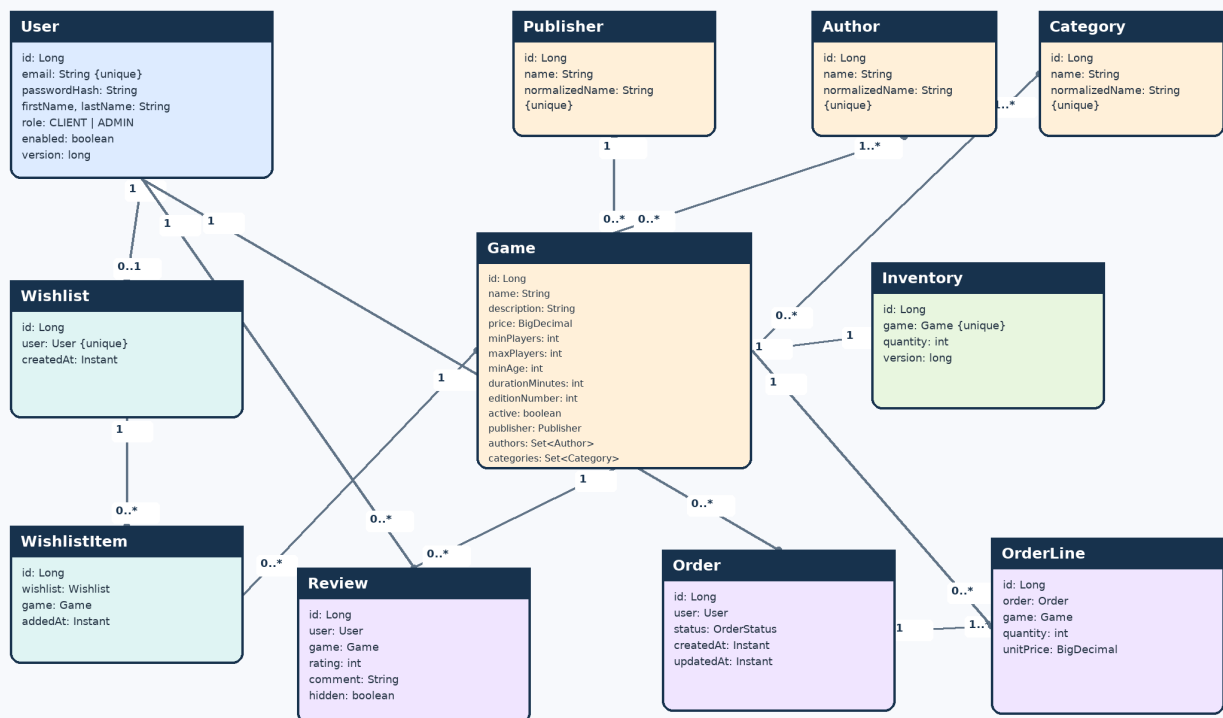
Responsabilités par composant

Composant	Responsabilité	Données manipulées	Exposition
Angular	Présentation et appels API	DTO JSON et JWT	Navigateur, hors dépôt
Spring Boot	Sécurité, validation, règles, transactions, orchestration	Utilisateurs, catalogue, commandes, interactions	Port hôte 8080
MySQL	Persistance transactionnelle	Schéma relationnel versionné	Réseau Docker interne
FastAPI	Préparation, entraînement et inférence KNN	Identifiants, notes, achats, scores	Réseau Docker interne
Volume modèle	Durabilité de l'artefact	Matrice, index, statistiques et métadonnées	Montage FastAPI uniquement

Modèle de données final

Modèle de classes persistant final

Les cardinalités reflètent les migrations Flyway V1 à V4 et les entités JPA.



Contraintes notables : email unique, une ligne par jeu et commande, un avis par utilisateur et jeu

Diagramme des classes persistantes et de leurs cardinalités

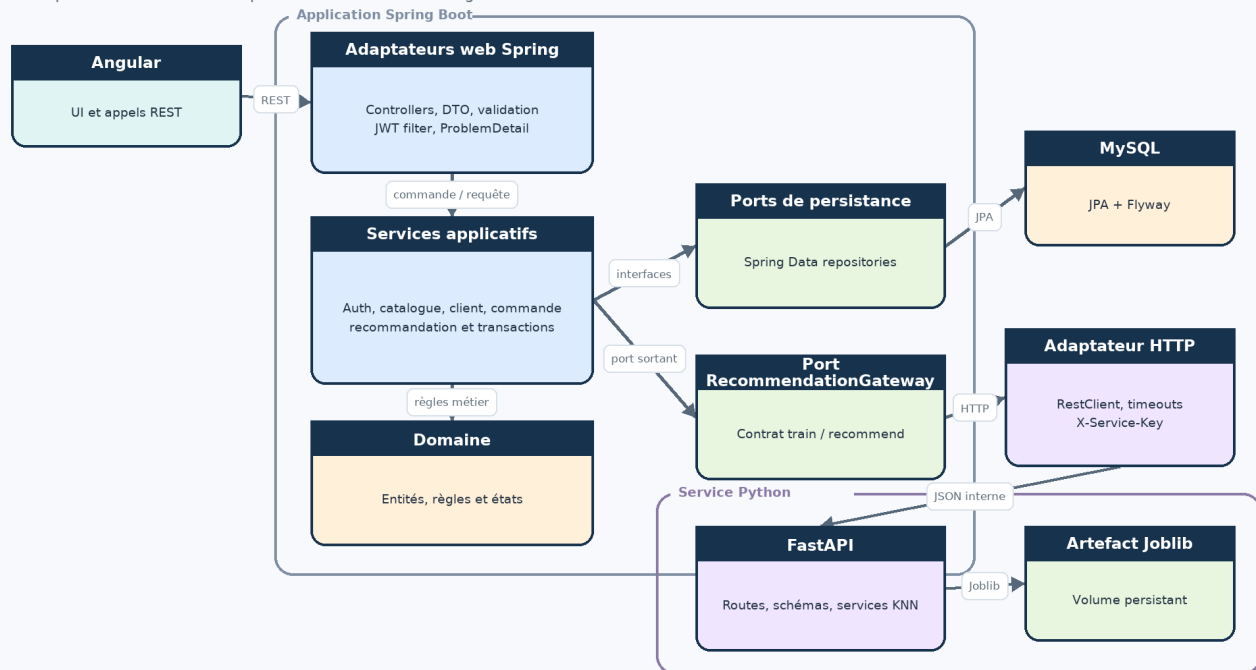
Le modèle sépare le catalogue, les interactions client et l'exécution des commandes. Game référence un éditeur et plusieurs auteurs et catégories. Un jeu possède une ligne de stock. Un utilisateur possède au plus une liste de souhaits, peut publier des avis et créer des commandes. Une commande comporte au moins une ligne ; le prix unitaire est mémorisé et le total est recalculé à partir de la quantité, ce qui préserve l'historique tarifaire.

Les tables de liaison rendent explicites les relations plusieurs-à-plusieurs entre jeux, auteurs et catégories. Les contraintes uniques évitent les doublons significatifs : email utilisateur, nom de référentiel, couple avis utilisateur-jeu, couple ligne commande-jeu et jeu de l'inventaire. Les cardinalités sont protégées à la fois par les mappings JPA et les contraintes créées par Flyway.

Composants et direction des dépendances

Composants, ports et adaptateurs

Les dépendances vont des adaptateurs vers les cas d'usage et le domaine.



Le port RecommendationGateway isole le métier Spring de HTTP et de FastAPI

Diagramme des composants Spring, des ports et des adaptateurs

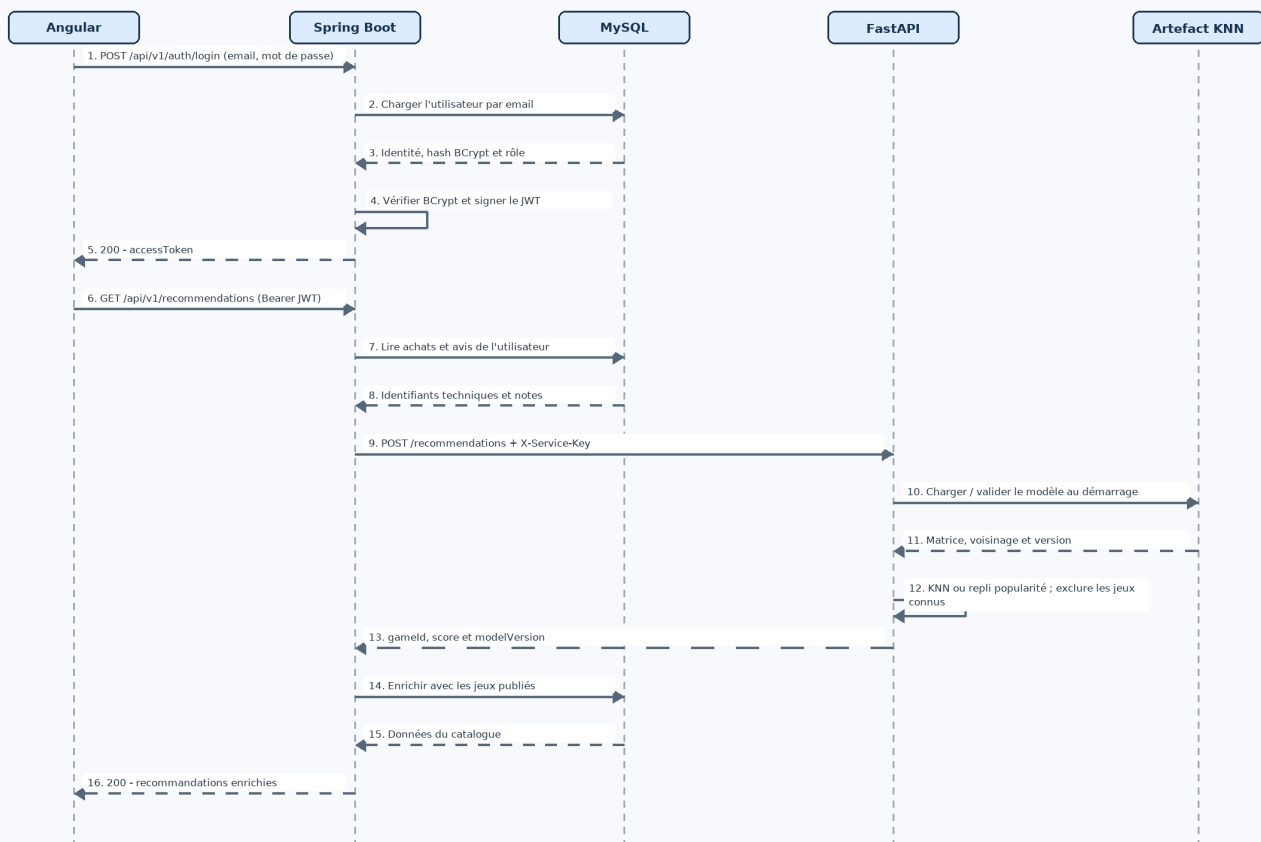
Les contrôleurs et filtres de sécurité sont des adaptateurs entrants. Ils convertissent HTTP en appels de cas d'usage, sans porter de SQL ni d'algorithme métier. Les services applicatifs gèrent l'orchestration et les frontières transactionnelles. Le domaine conserve les invariants. Les dépôts JPA et le client HTTP FastAPI sont des adaptateurs sortants.

RecommendationGateway constitue le port sortant le plus explicite. RecommendationService dépend de cette interface et non de RestClient ni de FastAPI. L'adaptateur FastApiRecommendationGateway porte les URI, la clé de service, les délais et la traduction des erreurs. Cette inversion permet de tester le service Java avec un double sans démarrer Python.

Séquence d'authentification et de recommandation

Séquence : authentification puis recommandation

Le JWT protège l'appel client ; la clé de service et le réseau interne protègent FastAPI.



FastAPI ne reçoit ni nom ni email ; une indisponibilité est traduite en ProblemDetail 503

Séquence complète de connexion puis consultation des recommandations

Le premier échange produit un JWT après vérification BCrypt. Le second réutilise ce jeton pour identifier le client et lire ses achats et avis. Spring transforme cet historique en interactions techniques, appelle FastAPI avec une clé de service, puis enrichit les identifiants recommandés avec les jeux encore publiés. Ainsi, l'algorithme ne devient jamais la source de vérité du catalogue.

Si le modèle est absent, invalide ou incompatible, FastAPI expose cet état par son contrôle de santé et refuse l'inférence. Spring traduit l'indisponibilité en réponse 503 au format ProblemDetail. Une réponse Python syntaxiquement valide mais incohérente est traduite en 502. Ces comportements empêchent une panne du moteur de recommandation de corrompre les données métier.

3. C.16 - Conception et qualité logicielle

Application des principes SOLID

Principe	Application concrète	Effet recherché
Responsabilité unique	Contrôleurs, services, entités, dépôts, sécurité et client FastAPI sont séparés	Compréhension et tests ciblés
Ouvert/fermé	Une nouvelle stratégie de recommandation peut implémenter le même gateway	Évolution sans réécrire les cas d'usage
Substitution	Les doubles de RecommendationGateway remplacent l'adaptateur HTTP en test	Contrats stables et tests rapides
Ségrégation des interfaces	Les dépôts et le gateway sont centrés sur des besoins précis	Moins de dépendances inutiles
Inversion des dépendances	Le service de recommandation dépend du port, pas du transport	Découplage Spring/FastAPI

Cette organisation n'est pas une architecture hexagonale exhaustive pour chaque module : Spring Data reste utilisé directement par plusieurs services applicatifs. Ce compromis réduit le volume de code tout en isolant la dépendance externe la plus volatile, le moteur Python.

DTO, validation et contrat HTTP

Les objets reçus et renvoyés par les contrôleurs sont des DTO dédiés. Les entités JPA ne sont donc pas sérialisées directement. Cette frontière évite de publier des champs internes comme le hash du mot de passe, la version d'optimistic locking ou les associations paresseuses. Elle limite également le couplage du client à la structure de la base.

Jakarta Validation porte les contraintes élémentaires : présence, formats, bornes, tailles et valeurs positives. Les contrôleurs déclenchent la validation et les services vérifient les règles nécessitant un état métier, par exemple la disponibilité du stock, la cohérence des joueurs ou une transition de commande autorisée. Le fichier OpenAPI versionné décrit les routes, schémas, statuts et exigences d'authentification.

Migrations et stratégie de schéma

Le paramètre Hibernate ddl-auto est réglé sur validate et open-in-view est désactivé. Hibernate vérifie donc le mapping sans modifier implicitement la base. Flyway est l'unique mécanisme d'évolution du schéma :

1. V1 crée les utilisateurs, les rôles et le verrou optimiste ;
2. V2 crée le catalogue, l'éditeur, les auteurs, les catégories et leurs liaisons ;
3. V3 ajoute l'inventaire, la liste de souhaits, les avis, les commandes et leurs lignes ;
4. V4 ajoute la modération par le champ hidden des avis.

La suite d'intégration démarre un vrai MySQL avec Testcontainers. Elle vérifie que les migrations s'appliquent et que les comportements JPA correspondent au moteur utilisé en production locale, plutôt qu'à une base embarquée présentant des différences de dialecte.

Transactions et concurrence

Les services applicatifs annotent explicitement les lectures et les écritures avec `@Transactional`. La création d'une commande constitue une unité atomique : validation des lignes, chargement du stock, calcul des montants, décrémentation de l'inventaire et enregistrement de la commande. Une erreur annule l'ensemble.

`InventoryRepository` utilise un verrou pessimiste en écriture lors de la réservation et `Inventory` possède aussi un champ `@Version`. Ce double mécanisme protège contre la vente concurrente du dernier exemplaire et contre des mises à jour perdues hors du flux de commande. Les transitions de statut sont centralisées afin d'interdire les retours incohérents.

Gestion des erreurs

`ApiExceptionHandler` transforme les exceptions en RFC 7807 `ProblemDetail`, avec un type `urn:gamesup:problem:<status>` et l'URI de la requête. La correspondance principale est la suivante :

Statut	Situation
400	Corps, paramètre ou contrainte de validation invalide
401	Identifiants absents ou invalides
403	Rôle insuffisant ou règle d'accès refusée
404	Ressource métier introuvable
409	Doublon, conflit d'état ou intégrité de données
422	Règle métier violée
502	Réponse incohérente du service FastAPI
503	Service de recommandation indisponible ou modèle non prêt

Cette convention rend les erreurs prévisibles pour Angular et empêche la remontée de traces techniques. Les timeouts de connexion et de lecture FastAPI sont configurables par variables d'environnement.

4. C.17 - Sécurité, tests et intégration continue

Sécurité de l'API Spring

Spring Security fonctionne sans session serveur. L'inscription et la connexion, la lecture du catalogue et la documentation OpenAPI sont publiques ; toutes les autres routes nécessitent un JWT valide. Les routes `/api/v1/admin/**` exigent le rôle ADMIN. Le rôle CLIENT couvre les fonctions personnelles.

Les mots de passe sont hachés par BCrypt et ne sont jamais renvoyés. Le JWT a un émetteur, une durée de vie et une clé injectés par configuration. CORS n'accepte que les origines déclarées. CSRF est désactivé parce que l'API est stateless et reçoit le jeton dans l'en-tête `Authorization`, sans cookie de session. Les secrets MySQL, JWT et FastAPI proviennent de `.env`, ignoré par Git ; `.env.example` ne contient que des valeurs de développement à remplacer.

FastAPI ajoute une seconde barrière : seuls les réseaux non publiquement routables autorisés peuvent appeler les routes sensibles, et une comparaison constante valide `X-Service-Key`. Le payload ne contient ni email ni nom. Il transporte `userId`, `gameId`, `rating` et le type de signal.

Stratégie de tests Spring

La stratégie respecte l'exigence de concentrer les tests automatisés fonctionnels sur le backend Spring :

- tests unitaires des règles de domaine et services applicatifs avec doubles de dépendances ;
- tests web des statuts, DTO, validations, authentification et autorisations ;
- tests de persistance et migrations sur MySQL Testcontainers ;
- tests d'intégration de l'adaptateur FastAPI par serveur HTTP simulé ;
- tests de concurrence et de transitions de commande ;
- test du contexte Spring comme filet de câblage, mais plus comme unique preuve.

Le job Python de la CI installe le package, compile les modules et importe l'application. Il s'agit d'un contrôle d'assemblage, pas d'une prétendue campagne fonctionnelle Python.

Résultat JaCoCo final

La commande de référence est :

```
cd Code/gamesUP
./mvnw clean verify
```

Indicateur	Résultat final
Tests Spring	113 exécutés, 0 échec, 0 erreur, 0 ignoré
Lignes	1 212 couvertes, 86 manquées, 1 298 analysées
Couverture de lignes	93,3744 %, arrondie à 93,37 %
Branches	175 couvertes, 53 manquées, soit 76,75 %
Seuil Maven	70 % des lignes au niveau du bundle
Classes exécutables JaCoCo	83 couvertes, 1 manquée

Les exclusions JaCoCo sont limitées au bootstrap GamesUpApplication, au package de configuration et aux DTO. Elles évitent que du code déclaratif déforme l'indicateur, sans exclure contrôleurs, services, sécurité, entités, dépôts ou adaptateurs. Le rapport local complet est généré dans Code/gamesUP/target/site/jacoco/index.html. Le fichier target/ est volontairement ignoré : il faut exécuter verify pour le recréer.

Le seuil n'est pas décoratif. Le plugin jacoco:check est lié à la phase Maven verify ; une campagne volontairement réduite au seul bootstrap a mesuré 18 % et a fait échouer le build, ce qui confirme le caractère bloquant du garde-fou.

Intégration continue

Le workflow .github/workflows/ci.yml comporte trois jobs indépendants : vérification Spring avec Java 21, compilation/import Python avec Python 3.11, puis validation et construction des images Docker Compose. Le job Spring lance verify et publie le répertoire JaCoCo HTML/XML comme artefact jacoco-report pendant 14 jours, y compris lorsque la vérification échoue.

5. C.18 - Système de recommandation

Données nécessaires et minimisation

Le filtrage collaboratif exploite les interactions explicites et implicites :

Signal	Source	Valeur utilisée
Note explicite	Avis utilisateur publié ou masqué côté métier	Moyenne si plusieurs signaux sont reçus pour un couple utilisateur-jeu
Achat implicite	Ligne de commande	Note synthétique 3,0

Signal	Source	Valeur utilisée
Identité technique	Identifiants internes	Construction des axes utilisateur et jeu

Les noms, emails, mots de passe et commentaires ne sont pas nécessaires à l'algorithme et ne quittent pas Spring. La préparation déduplique chaque couple utilisateur-jeu. L'entraînement refuse un corpus de moins de deux interactions, deux utilisateurs ou deux jeux, car un voisinage n'aurait alors pas de sens.

Entraînement KNN

Le service construit une matrice creuse CSR utilisateur-jeu, puis la transpose afin de comparer les jeux selon leurs vecteurs d'utilisateurs. `sklearn.neighbors.NearestNeighbors` utilise la distance cosinus avec l'algorithme brute. Cette combinaison convient à une matrice creuse et évite une densification coûteuse.

Pour un utilisateur connu, les notes strictement supérieures à 3,0 définissent les préférences positives. Les voisins des jeux appréciés reçoivent un score pondéré par la préférence normalisée. Les jeux déjà connus de l'utilisateur sont exclus, les résultats sont triés par score décroissant puis identifiant pour assurer un ordre stable.

Démarrage à froid et repli

Un utilisateur inconnu, ou connu sans préférence positive, reçoit un classement de popularité. Ce classement emploie un score bayésien réduit vers la moyenne globale : il évite qu'un jeu noté une seule fois domine un jeu régulièrement apprécié. Le même repli complète une liste KNN trop courte.

Le démarrage à froid d'un nouveau jeu reste imparfait : sans interaction, il ne dispose pas de vecteur collaboratif. Une approche hybride exploitant catégories, auteur, éditeur, âge et nombre de joueurs serait nécessaire pour l'intégrer immédiatement.

Persistance et cycle de vie du modèle

L'artefact Joblib enregistre la version de schéma, une version de modèle issue du contenu, l'horodatage, la version scikit-learn, les mappings utilisateurs/jeux, la matrice CSR et les statistiques de popularité. L'écriture est atomique. Au démarrage, FastAPI classe son état en `NOT_TRAINED`, `READY`, `INVALID` ou `INCOMPATIBLE` après validation de l'artefact.

L'entraînement est déclenché par `POST /api/v1/admin/recommendations/train`. Spring agrège les achats et avis en base puis envoie le corpus à FastAPI. Ce choix garantit qu'un administrateur contrôle une opération coûteuse et que MySQL reste la source de vérité.

Limites et stratégie de réentraînement

Le modèle actuel est adapté à une démonstration d'architecture, pas à une exploitation à grande échelle. Ses limites sont la rareté des données au lancement, le biais de popularité, l'absence de contexte temporel, l'absence de diversité contrôlée et l'absence d'évaluation hors ligne ou en ligne. Les données d'exemple ne préjugent pas de la distribution réelle.

La trajectoire recommandée est la suivante :

1. automatiser un entraînement périodique en tâche de fond et conserver le dernier artefact valide ;
2. imposer un support minimal par utilisateur et par jeu, puis isoler un jeu de validation temporel ;
3. mesurer `precision@k`, `recall@k`, `NDCG`, couverture catalogue et diversité ;
4. surveiller fraîcheur, dérive, latence, taux de repli et taux d'erreur ;
5. ajouter des caractéristiques de contenu et mener une expérimentation A/B avant généralisation.

6. Démonstration reproductible avec Docker Compose

Préparation et démarrage

Depuis la racine du dépôt, créer la configuration locale, remplacer les quatre secrets de développement puis démarrer les services :

```
cp .env.example .env
# Modifier MYSQL_PASSWORD, MYSQL_ROOT_PASSWORD, JWT_SECRET et FASTAPI_SERVICE_KEY.
docker compose build --pull --no-cache
docker compose up --detach
docker compose ps
```

MySQL doit être healthy, puis FastAPI, puis Spring. Flyway s'exécute au démarrage de Spring. Les commandes suivantes vérifient les trois couches sans exposer MySQL ni FastAPI sur l'hôte :

```
curl --fail 'http://localhost:8080/api/v1/games?size=1'
docker compose exec fastapi python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8000/health').read().decode())"
docker compose exec mysql sh -c 'mysql -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE" -e "SELECT version, description, success FROM flyway_schema_history ORDER BY installed_rank"'
```

Parcours authentifié

Le script suivant crée un client unique, extrait son JWT et appelle son profil :

```
./scripts/demo.sh
```

Le même parcours peut être réalisé manuellement :

```
curl --fail --request POST 'http://localhost:8080/api/v1/auth/register' \
  --header 'Content-Type: application/json' \
  --data '{"email":"demo@example.test","password":"Demo-password-2026!","firstName":"Demo","lastName":"GamesUP"}'

curl --fail --request POST 'http://localhost:8080/api/v1/auth/login' \
  --header 'Content-Type: application/json' \
  --data '{"email":"demo@example.test","password":"Demo-password-2026!"}'

curl --fail 'http://localhost:8080/api/v1/users/me' \
  --header 'Authorization: Bearer <JETON_RETOURNE>'
```

La recommandation requiert un corpus d'au moins deux utilisateurs, deux jeux et deux interactions, ainsi qu'un compte ADMIN pour lancer l'entraînement. Ces données ne sont volontairement pas semées avec des identifiants par défaut. Dans une base de démonstration préalablement alimentée, l'administrateur appelle :

```
curl --fail --request POST 'http://localhost:8080/api/v1/admin/recommendations/train' \
  --header 'Authorization: Bearer <JETON_ADMIN>'

curl --fail 'http://localhost:8080/api/v1/recommendations?limit=5' \
  --header 'Authorization: Bearer <JETON_CLIENT>'
```

Avant l'entraînement, la seconde commande doit produire un 503 documenté plutôt qu'une liste inventée. L'arrêt normal conserve les volumes avec `docker compose down`. La commande `docker compose down --volumes` est destructive et ne doit être utilisée que pour réinitialiser explicitement les données locales et l'artefact.

7. Regard critique et améliorations

Choix et compromis assumés

- L'API Spring demeure l'unique façade publique. Cette simplicité réduit la surface d'attaque mais en fait un point de passage obligatoire ; une montée en charge demanderait réplication et équilibrage.
- Les repositories Spring Data sont employés directement par certains services. Le coût d'une abstraction complète n'était pas justifié pour tous les agrégats, contrairement au port FastAPI qui protège une dépendance distante et évolutive.
- Le verrou pessimiste sécurise le stock avec une sémantique claire, au prix d'une contention possible lors de pics sur un jeu populaire. Une réservation asynchrone ou un compteur atomique serait à étudier selon la charge réelle.
- Le déclenchement manuel du réentraînement est fiable et observable pour la démonstration, mais insuffisant pour des données qui évoluent en continu.
- La couverture de 93,37 % est une preuve utile, pas une garantie absolue. La qualité dépend aussi de la pertinence des assertions, des cas de concurrence et des scénarios d'échec.

Risques résiduels prioritaires

1. Aucun frontend Angular n'est livré dans ce dépôt : l'intégration d'interface reste à construire et à tester contre OpenAPI.
2. La rotation des clés JWT et de service, la révocation des jetons et la gestion centralisée des secrets ne sont pas implémentées.
3. Le moteur KNN ne dispose pas encore d'un protocole d'évaluation métier ni d'un jeu de données représentatif.
4. Les sauvegardes MySQL et du volume modèle, la restauration et la reprise après sinistre doivent être documentées et exercées.
5. L'observabilité doit être complétée par métriques, traces distribuées, corrélation des requêtes et alertes sur les états du modèle.

Améliorations recommandées

À court terme, construire le client Angular depuis le contrat OpenAPI, ajouter des tests de contrat consommateur et mettre les secrets dans un coffre. À moyen terme, ajouter OpenTelemetry, limiter le débit des routes d'authentification, automatiser le réentraînement et mesurer la qualité du classement. À plus long terme, tester une recommandation hybride, mettre en place une stratégie de déploiement progressif et valider les objectifs de reprise des données.

8. Traçabilité des preuves

Index des preuves

Élément	Source versionnée ou commande
Contrat HTTP	Code/gamesUP/docs/openapi.yaml
Entités et services Spring	Code/gamesUP/src/main/java/com/gamesup/api/
Migrations	Code/gamesUP/src/main/resources/db/migration/
Tests Spring	Code/gamesUP/src/test/java/com/gamesup/api/
Configuration JaCoCo	Code/gamesUP/pom.xml
Rapport JaCoCo local	Code/gamesUP/target/site/jacoco/index.html après verify
Workflow CI	.github/workflows/ci.yml
Service KNN	Code/CodeApiPython/gamesup_api/
Déploiement	compose.yaml, .env.example, docs/docker.md

Conclusion

La solution finale répond aux objectifs backend de l'étude de cas : architecture lisible, persistance Hibernate, contrat DTO, sécurité à deux rôles, tests Spring avec seuil bloquant et recommandation KNN réellement connectée. Les limites restent explicites : Angular n'est pas livré, le corpus réel manque encore et l'exploitation demanderait une gestion avancée des secrets, de l'observabilité et une évaluation ML continue. Chaque affirmation de ce rapport renvoie à une preuve versionnée dans le dépôt : le code, les migrations, les tests et le workflow d'intégration continue.